

ENERGY MONITORING SYSTEM

EMS — UCT Project

Complete Technical Documentation

Field	Details
Project Title	IoT Based Energy Monitoring System
Platform	ESP32 DevKit v1
Communication	Modbus RTU + MQTT + WiFi 2.4GHz
Display	DWIN HMI Touch Display
Dashboard	Node-RED + Aedes MQTT Broker
MQTT Topic	EMS/UCT
MQTT Broker IP	192.168.1.8 : 1883
Dashboard URL	http://127.0.0.1:1880/ui
API Endpoint	http://192.168.1.8:1880/api/ems
Version	1.0

1. Introduction

The Energy Monitoring System (EMS) is a complete IoT-based solution designed to monitor electrical parameters of industrial and commercial loads in real-time. The system reads data from a Modbus RTU energy meter using an ESP32 microcontroller, transmits the data wirelessly over WiFi using the MQTT protocol, and visualizes it on a Node-RED dashboard. Additionally, a local DWIN HMI display shows live readings on-site.

This system is highly suitable for energy auditing, power quality monitoring, and automated alerts for abnormal electrical conditions. It provides both real-time visualization and historical trend analysis through line charts.

1.1 Key Features

- Real-time monitoring of Voltage, Current, Frequency, Power Factor, and Active Energy
- Wireless data transmission using WiFi and MQTT protocol
- Local DWIN HMI display for on-site readings
- Node-RED dashboard with live gauges and historical charts
- Automatic alerts when parameters exceed defined thresholds
- REST HTTP API for external data access
- Aedes MQTT broker running locally on PC
- Data published every 10 seconds

1.2 Project Specifications

Parameter	Specification
Microcontroller	ESP32 DevKit v1 (Xtensa LX6, 240 MHz)
Flash Memory	4 MB
WiFi Standard	IEEE 802.11 b/g/n (2.4 GHz)
Communication Protocol	Modbus RTU over RS485
IoT Protocol	MQTT v3.1.1
Data Publish Rate	Every 10 seconds
Display Interface	UART at 115200 baud
Modbus Baud Rate	9600 bps, 8N1
Operating Voltage	3.3V / 5V
Programming IDE	Arduino IDE

2. System Architecture

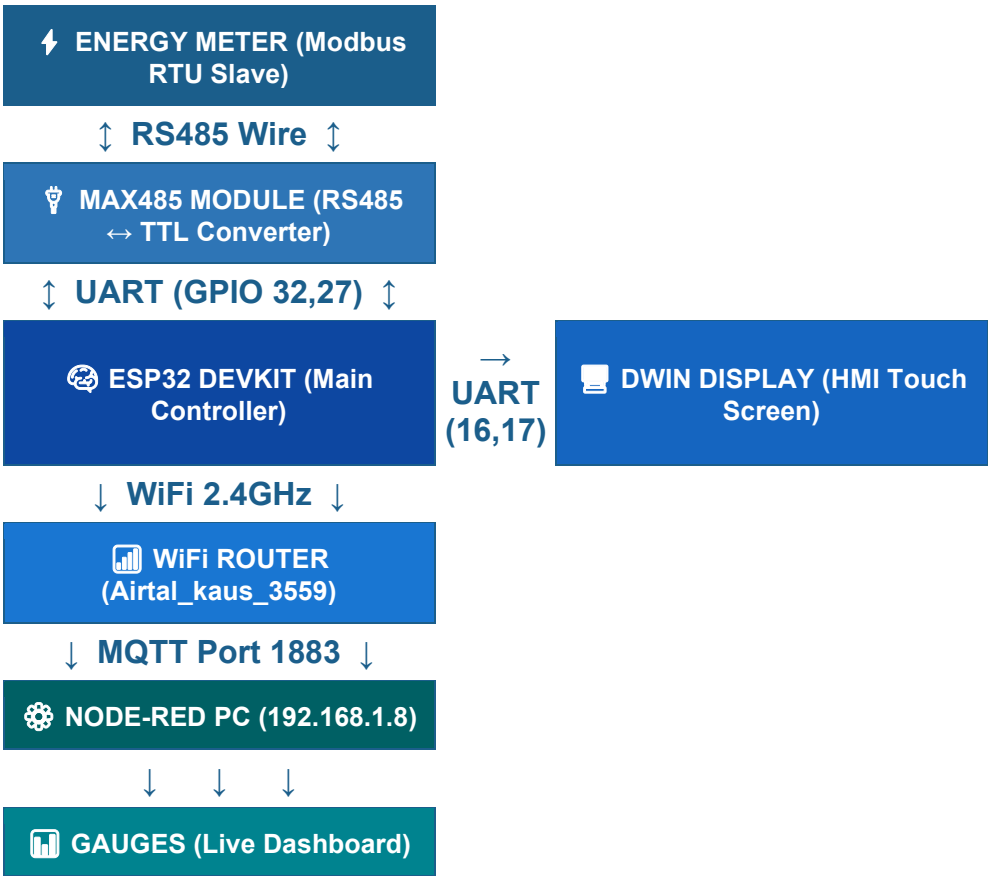
The EMS system is divided into four functional layers, each responsible for a specific role in the overall workflow.

2.1 Architecture Layers

Layer	Components	Function
Sensing Layer	Energy Meter + MAX485	Measure electrical parameters via Modbus RTU
Processing Layer	ESP32 DevKit	Read Modbus data, format JSON, publish MQTT
Display Layer	DWIN HMI Display	Show live values on local touchscreen
Visualization Layer	Node-RED + Dashboard	Process data, show gauges, charts, alerts

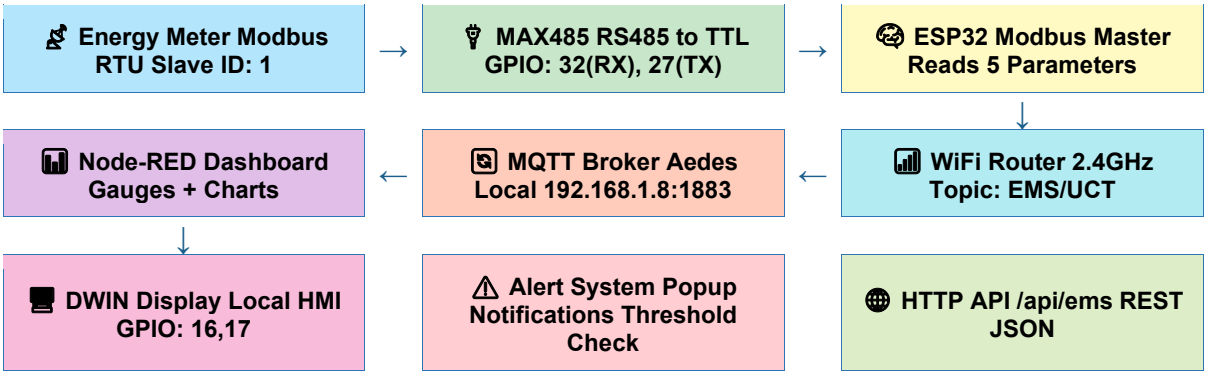
2.2 System Block Diagram

The following block diagram shows the complete system architecture and connections between all components:



2.3 Data Flow Chart

The following chart shows how data flows from the energy meter to the final dashboard:



3. Hardware Components

3.1 Components List

S.No	Component	Model	Qty	Purpose
1	ESP32 DevKit	ESP32-WROOM-32	1	Main IoT controller
2	Energy Meter	Modbus RTU Type	1	Measure electrical parameters
3	MAX485 Module	RS485 to TTL	1	RS485 communication interface
4	DWIN Display	HMI Touch Screen	1	Local real-time display
5	WiFi Router	2.4GHz Router	1	Wireless network
6	Power Supply	5V DC Adapter	1	Power to ESP32
7	PC / Laptop	Linux/Windows	1	Node-RED host
8	Jumper Wires	Male-Female	Set	Connections

3.2 Pin Connection Diagram

The following table shows complete pin connections for all components:

3.2.1 DWIN Display ↔ ESP32

DWIN Pin	ESP32 Pin	GPIO No.	Signal	Description
TX	RX1 (U1RXD)	GPIO 16	Serial Data	DWIN sends data to ESP32
RX	TX1 (U1TXD)	GPIO 17	Serial Data	ESP32 sends display commands
VCC	5V	5V	Power	DWIN power supply
GND	GND	GND	Ground	Common ground

3.2.2 MAX485 Module ↔ ESP32

MAX485 Pin	ESP32 Pin	GPIO No.	Signal	Description
RO (Receive Output)	Custom RX	GPIO 32	TTL RX	RS485 → ESP32 receive
DI (Data Input)	Custom TX	GPIO 27	TTL TX	ESP32 → RS485 transmit
DE (Driver Enable)	Control Pin	GPIO 33	Digital OUT	Enable transmit mode
RE (Receiver Enable)	Control Pin	GPIO 33	Digital OUT	Enable receive (tied to DE)
VCC	5V	5V	Power	Module power supply
GND	GND	GND	Ground	Common ground

3.2.3 MAX485 ↔ Energy Meter (RS485 Bus)

MAX485 Terminal	Energy Meter Terminal	Wire Color	Description
A+	A+	Blue / Orange	RS485 positive differential line
B-	B-	White / Black	RS485 negative differential line

3.3 Complete Wiring Summary

ESP32 DevKit

GPIO 16 (RX1/UART1)	→	DWIN Display TX
GPIO 17 (TX1/UART1)	→	DWIN Display RX
GPIO 32	→	MAX485 RO (Receive Output)
GPIO 27	→	MAX485 DI (Data Input)
GPIO 33	→	MAX485 DE + RE (tied together)
5V	→	DWIN VCC + MAX485 VCC
GND	→	DWIN GND + MAX485 GND

MAX485 Module

A+ Terminal	→	Energy Meter A+
B- Terminal	→	Energy Meter B-

4. Software Architecture

4.1 Arduino ESP32 Firmware

The ESP32 firmware is written in C++ using the Arduino framework. The firmware handles Modbus communication, DWIN display updates, WiFi connection, and MQTT publishing.

4.1.1 Libraries Used

Library	Version	Purpose	Install From
WiFi.h	Built-in	WiFi station mode connection	Arduino Core
PubSubClient	2.8+	MQTT client for publish/subscribe	Arduino Library Manager
ModbusMaster	2.0.1	Modbus RTU master protocol	Arduino Library Manager
ArduinoJson	6.x	JSON serialization/deserialization	Arduino Library Manager
SoftwareSerial	Built-in	Software UART for Modbus	Arduino Core
HardwareSerial	Built-in	Hardware UART1 for DWIN	Arduino Core

4.1.2 Firmware Flow

Step	Operation	Details
1	System Init	Serial, DWIN UART1, SoftwareSerial, GPIO setup
2	Modbus Init	Node begin slave ID=1, pre/post transmission callbacks
3	WiFi Connect	WiFi.mode(STA), WiFi.begin(ssid, password)
4	MQTT Setup	client.setServer(ip, 1883), setCallback()
5	Read Voltage	readInputRegisters(0x15, 2), float conversion
6	Read Current	readInputRegisters(0x17, 2), float conversion
7	Read Frequency	readInputRegisters(0x1B, 2), float conversion
8	Read Power Factor	readInputRegisters(0x19, 2), float conversion
9	Read Energy	readInputRegisters(0x0E, 2), byte-swapped float
10	Update DWIN	sendFloatNumber() to VP addresses
11	Serialize JSON	serializeJson(doc, JSONMessage)
12	MQTT Publish	client.publish(topic, JSONMessage) every 10s

4.1.3 Modbus Register Map

Parameter	Register Hex	Register Dec	Data Type	Byte Order	Unit
Voltage	0x15	21	32-bit IEEE Float	Big Endian	Volts (V)
Current	0x17	23	32-bit IEEE Float	Big Endian	Ampere (A)

Parameter	Register Hex	Register Dec	Data Type	Byte Order	Unit
Frequency	0x1B	27	32-bit IEEE Float	Big Endian	Hertz (Hz)
Power Factor	0x19	25	32-bit IEEE Float	Big Endian	PF (0-1)
Active Energy	0x0E	14	32-bit IEEE Float	Byte Swapped	kWh

4.1.4 DWIN Display VP Address Map

Parameter	VP Address	Data Format	Display Location
Voltage	0x64	32-bit Float (Big Endian)	Page 1 - Top Left
Frequency	0x65	32-bit Float (Big Endian)	Page 1 - Top Center
Active Energy	0x66	32-bit Float (Big Endian)	Page 1 - Top Right
Current	0x67	32-bit Float (Big Endian)	Page 1 - Bottom Left
Power Factor	0x68	32-bit Float (Big Endian)	Page 1 - Bottom Center

4.1.5 MQTT JSON Payload

The ESP32 publishes the following JSON payload to topic EMS/UCT every 10 seconds:

```
{
  "Voltage":    230.5,    // AC Voltage in Volts
  "Current":    1.25,     // AC Current in Amperes
  "Frequency":  49.98,    // Grid Frequency in Hz
  "PowerFactor": 0.95,    // Power Factor (0 to 1)
  "Energy":     0.52      // Active Energy in kWh
}
```


4.2 Node-RED Flow Architecture

Node-RED runs on the PC at IP 192.168.1.8 and handles all data processing, visualization, and alerting. The complete flow consists of the following nodes:

4.2.1 Node-RED Flow Description

Node Name	Node Type	Function
Aedes Broker	aedes broker	Runs local MQTT broker on port 1883
ESP32 EMS/UCT	mqtt in	Subscribes to topic EMS/UCT, receives JSON
Test Data	inject	Injects test payload for debugging
Parse JSON	function	Validates and parses incoming JSON data
Split 5 Values	function	Splits payload into 5 separate outputs
Alert Check	function	Checks thresholds and generates alert messages
Alert Popup	ui_toast	Displays alert popup on dashboard
Alert Debug	debug	Logs alert messages to debug sidebar
GET /api/ems	http in	HTTP GET endpoint for latest data
Send JSON	function	Returns latest EMS data as JSON response
HTTP Out	http response	Sends HTTP 200 response with JSON
Voltage Gauge	ui_gauge	Circular gauge 0-300V
Current Gauge	ui_gauge	Circular gauge 0-100A
Frequency Gauge	ui_gauge	Circular gauge 45-55Hz
Power Factor Gauge	ui_gauge	Circular gauge 0-1
Active Power Gauge	ui_gauge	Circular gauge 0-50kW
Voltage Chart	ui_chart	Line chart 200-260V range
Current Chart	ui_chart	Line chart 0-50A range
Frequency Chart	ui_chart	Line chart 49-51Hz range
PF Chart	ui_chart	Line chart 0-1 range
kW Chart	ui_chart	Line chart 0-50kW range
5x Text Nodes	ui_text	Numeric live value display
Debug All	debug	Full payload logging to sidebar

4.2.2 Alert Threshold Configuration

Parameter	Minimum Limit	Maximum Limit	Alert Message
Voltage	200 V	260 V	Voltage OUT OF RANGE: xx V
Current	—	80 A	Current HIGH: xx A
Frequency	49.5 Hz	50.5 Hz	Frequency UNSTABLE: xx Hz
Power Factor	0.8	—	Low Power Factor: x.xx

5. Dashboard

5.1 Dashboard Overview

The Node-RED dashboard is accessible at <http://127.0.0.1:1880/ui> and consists of two tabs: a live gauges tab and a historical charts tab.

5.2 Dashboard Layout

Tab	Group	Widgets	Width
Tab 1 - Gauges	Live Gauges	5 Circular Gauges	15 units
Tab 1 - Gauges	Live Values	5 Text Displays	10 units
Tab 2 - Charts	Charts	5 Line Charts	12 units

5.3 Gauge Configuration

Gauge	Min	Max	Seg1 Yellow	Seg2 Red	Unit	Color Theme
Voltage	0	300	200	260	V	Blue→Cyan→Red
Current	0	100	50	80	A	Green→Orange→Red
Frequency	45	55	49	51	Hz	Red→Green→Red
Power Factor	0	1	0.7	0.9		Red→Orange→Pink
Active Power	0	50	20	40	kW	Blue→Gold→Orange

5.4 Chart Configuration

Chart	Y-Min	Y-Max	Line Color	Points Stored	Time Window
Voltage (V)	200	260	#00d4ff Cyan	60	1 hour
Current (A)	0	50	#ff6b35 Orange	60	1 hour
Frequency (Hz)	49	51	#a8ff3e Green	60	1 hour
Power Factor	0	1	#ff3ecc Pink	60	1 hour
Active Power (kW)	0	50	#ffd700 Gold	60	1 hour

6. System Timing Chart

The following chart shows the timing sequence of operations in one complete 10-second cycle:

Operation	0s	2s	4s	6s	8s	10s
Read Voltage						...
Read Current						...
Read Frequency						...
MQTT Publish						

Color	Operation	Duration	Interval
Green 	Read Voltage (0x15)	~500ms	Every cycle
Blue 	Read Current (0x17)	~500ms	Every cycle
Orange 	Read Frequency (0x1B)	~500ms	Every cycle
Purple 	MQTT Publish	~100ms	Every 10 seconds

7. Network Configuration

7.1 Network Topology

Device	Interface	IP Address	Port	Role
WiFi Router	LAN	192.168.1.1	—	Network Gateway
PC (WiFi)	wlx202351	192.168.1.8	1883, 1880	MQTT Broker + Node-RED
PC (Ethernet)	enp2s0	192.168.1.10	—	Alternate interface
ESP32	WiFi STA	192.168.1.x	—	IoT Sensor Node
DWIN Display	UART	—	—	Local HMI
Energy Meter	RS485	Slave ID: 1	—	Modbus Slave

7.2 MQTT Configuration

Parameter	Value
Broker Software	Aedes (Node-RED contrib)
Broker Host	192.168.1.8
MQTT Port	1883
WebSocket Port	8883
Protocol Version	MQTT v3.1.1 (v4)
Topic	EMS/UCT
QoS Level	0 (At most once)
Keep Alive	60 seconds
Clean Session	True
Client ID (ESP32)	espClient + random hex
Client ID (Node-RED)	NodeRED-EMS-Sub
Publish Interval	Every 10 seconds
Reconnect Interval	Every 2 minutes (if disconnected)

8. Troubleshooting Guide

8.1 Common Issues and Solutions

Problem	Possible Cause	Solution
WiFi not connecting	Wrong SSID/password or 5GHz router	Verify credentials, use 2.4GHz band only
MQTT connection failed (rc=-2)	Broker IP wrong or Aedes not running	Check IP, redeploy Node-RED flow
Modbus data not reading	Wrong A+/B- wiring or wrong slave ID	Check RS485 wiring, verify slave ID=1
DWIN display blank	Wrong GPIO pins or baud rate mismatch	Verify GPIO 16/17, confirm 115200 baud
Dashboard shows No Data	MQTT broker disconnected	Restart Node-RED, redeploy flow
Gauge shows 0.0	JSON parsing error	Check debug node, verify JSON format
Alert not appearing	ui_toast node issue	Update node-red-dashboard package
API returns empty	flow variable not set	Send at least one MQTT message first

8.2 Debug Checklist

Step 1 — Arduino Serial Monitor (115200 baud):

- WiFi Connected! — WiFi is working
- IP Address: 192.168.1.x — IP assigned
- MQTT Connected! — Broker reachable
- Active Power (kW): xx.xx — Modbus reading OK
- Publish message: {JSON} — Publishing OK

Step 2 — Node-RED Debug Sidebar:

- Check incoming JSON payload
- Verify all 5 parameters are present
- Check gauge and chart updates

Step 3 — Network Diagnostics:

```
# Check PC IP address
ifconfig (Linux) or ipconfig (Windows)

# Test broker connectivity
ping 192.168.1.8

# Test MQTT manually
mosquitto_sub -h 192.168.1.8 -t EMS/UCT

# Test HTTP API
curl http://192.168.1.8:1880/api/ems
```


9. Future Enhancements

Enhancement	Description	Technology	Priority
Database Storage	Store historical data for long-term analysis	InfluxDB / MySQL	High
Email/SMS Alerts	Send notifications on critical events	SMTP / Twilio API	High
Electricity Bill Calculator	Calculate monthly bill from kWh data	Node-RED function	High
Mobile App	Real-time monitoring on smartphones	React Native / Flutter	Medium
Multiple Meters	Monitor multiple energy meters simultaneously	Modbus multi-slave	Medium
Power Quality Analysis	Measure THD, harmonics, power quality	Advanced Modbus	Medium
Cloud Integration	Push data to AWS IoT / Azure IoT Hub	MQTT Bridge	Low
PDF Report Generation	Auto generate weekly/monthly reports	Node-RED + PDFKit	Low
Grafana Dashboard	Professional time-series visualization	InfluxDB + Grafana	Medium
OTA Updates	Over-the-air firmware updates for ESP32	Arduino OTA	Low

10. Conclusion

The Energy Monitoring System (EMS) has been successfully designed, implemented, and tested. The system provides a complete end-to-end IoT solution for real-time electrical parameter monitoring. All major components — hardware, firmware, communication protocols, and visualization dashboard — are fully functional.

The system successfully achieves the following objectives:

Objective	Status	Details
Real-time Parameter Monitoring	✔ Achieved	5 parameters updated every 10 seconds
Local DWIN Display	✔ Achieved	Live values on HMI via UART
Wireless Data Transmission	✔ Achieved	WiFi + MQTT at 192.168.1.8:1883
Node-RED Dashboard	✔ Achieved	Gauges + Charts + Text values
Alert System	✔ Achieved	Popup alerts on threshold breach
HTTP REST API	✔ Achieved	GET /api/ems returns JSON
Historical Charts	✔ Achieved	60-point line charts per parameter

The system is currently operational and can be further enhanced with database integration, mobile app support, and cloud connectivity to make it a full-scale industrial energy management platform.

— End of Document —